

MetaRAG-Test: Engineering Case Study

Standalone engineering case study for metamorphic and groundedness testing of RAG systems

Repository: <https://github.com/pthanureddy/metarag-test>

Report version: July 2026

Abstract

Retrieval-augmented generation systems are difficult to test because correctness depends on both answer quality and evidence quality. A generated answer may be fluent but unsupported by retrieved context. A semantically correct answer may use wording that differs from a gold answer. A repeated run may produce a different but still acceptable answer. These properties make exact string assertions too weak for realistic RAG testing.

MetaRAG-Test is a standalone engineering case study that implements a compact but reproducible test harness for RAG behavior. It uses a public NIST AI Risk Management Framework study corpus, seed question-answer pairs, a deterministic retrieval pipeline, paraphrase-based metamorphic tests, k-run consistency scoring, a groundedness oracle, corrupted-context validation, and a RAGAS-style faithfulness comparison. The public CI path uses lightweight deterministic fallbacks so the repository can be cloned and tested without model keys or heavyweight downloads.

The current evaluation contains 20 seed questions and 60 paraphrase checks. The baseline run reports 100.00 percent groundedness accuracy on clean retrieved context, 100.00 percent hallucination catch rate on deliberately corrupted context, 0.00 percent metamorphic violation rate, 0.00 mean flakiness variance, and 100.00 percent agreement with the faithfulness proxy. These results should be interpreted as a validation of the harness mechanics over a controlled corpus, not as a broad benchmark for all RAG systems.

This case study documents the problem, design, implementation, evaluation method, current results, limitations, and reproduction steps. It positions the project as an engineering artifact for testing evidence-grounded language generation.

Table of Contents

1. Problem framing
2. Background and related work
3. System goals and non-goals
4. Corpus and seed question design
5. Retrieval and answer pipeline
6. Groundedness oracle
7. Metamorphic test design
8. Consistency scoring
9. Corrupted-context validation
10. Results and interpretation
11. Reproducibility and operations
12. Limitations, risks, and future work
13. References

1. Problem Framing

RAG systems combine retrieval with generation. Retrieval supplies evidence. Generation turns that evidence into a useful answer. This architecture is powerful, but it changes how testing should work. The same question can have several valid answers. Two answers can be textually different and semantically equivalent. A fluent answer can be wrong because it is not grounded in retrieved context. A short answer can be correct but incomplete. These properties make exact-match testing brittle.

The key testing question is not only "did the system answer?" The stronger questions are: did it retrieve relevant context, did the answer stay faithful to that context, did paraphrasing preserve meaning, and did repeated runs remain stable enough for the intended use? RAG testing therefore needs test oracles that measure groundedness and semantic consistency rather than only string equality.

MetaRAG-Test frames the problem as an engineering harness. It uses a known public corpus, a fixed seed bank, reproducible retrieval, generated paraphrases, controlled context corruption, and quantitative metrics. The implementation is intentionally small so the behavior can be audited. The system is not a production search product. It is a test harness for studying RAG failure modes.

The project also makes an important IP boundary explicit. The corpus is a public NIST AI RMF study corpus. There is no employer data, customer data, private document corpus, or proprietary retrieval logic. This boundary makes the test harness safe to publish and reuse.

RAG testing challenge	Why exact matching fails	Harness response
Multiple correct phrasings	Text can differ while meaning remains	Semantic similarity
Unsupported fluent answers	Fluency hides missing evidence	Groundedness oracle
Retrieval mistakes	Answer may cite wrong context	Corrupted-context validation
Non-deterministic outputs	Repeated runs vary	k-run consistency
Question paraphrases	Same intent has many forms	Metamorphic tests

2. Background and Related Work

RAG was introduced as a way to combine parametric generation with non-parametric retrieved memory for knowledge-intensive tasks [1]. The main benefit is that generated answers can be conditioned on retrieved passages rather than relying only on model weights. That same structure creates an evaluation burden: the system must be judged on retrieval quality, answer quality, and faithfulness to context.

RAGAS provides a practical evaluation framework for RAG systems, including metrics such as faithfulness, answer relevancy, context precision, and context recall [2], [8]. MetaRAG-Test does not attempt to replace RAGAS. Instead, it builds a smaller testing harness that exposes the same kinds of concerns in ordinary test cases and compares local proxy decisions with RAGAS-style faithfulness logic.

Metamorphic testing is useful when a system has no single obvious expected output. METAL applies metamorphic testing ideas to large language model qualities, using relations between transformed inputs and expected output properties [3]. MetaRAG-Test adapts that idea to RAG: paraphrasing a question should preserve the answer meaning if retrieval and generation are robust.

Semantic comparison relies on sentence embeddings when available. Sentence-BERT showed that sentence embeddings can support efficient semantic similarity comparison [4]. For groundedness, the project supports a local natural language inference path. The SNLI dataset and related NLI work frame entailment as a relationship between a premise and a hypothesis [5]. In this harness, retrieved context is the premise and the generated answer is the hypothesis.

The corpus is based on public NIST AI Risk Management Framework concepts [6]. This provides a realistic governance and risk-management domain while remaining safe for public reproduction. Optional OpenAI judging can be enabled through a key, but the public CI path avoids secrets.

3. System Goals and Non-goals

The first goal is to make RAG testing reproducible. The corpus, seed questions, retrieval logic, answer fallback, metrics, and report generation are all stored in the repository. A developer can run the test suite without a hosted model key. This matters because model-backed demos can be hard to inspect if the data and prompts are hidden.

The second goal is to test properties, not exact strings. The harness measures groundedness, paraphrase invariance, repeated-run stability, hallucination catch rate, and faithfulness-proxy agreement. These metrics align with how RAG systems fail in practice.

The third goal is to use deliberate negative controls. Corrupted-context tests swap relevant context for unrelated context while keeping the answer constant. A useful groundedness oracle should flag those cases as unsupported. This is a stronger validation than only checking that clean cases pass.

The fourth goal is to keep heavyweight integrations optional. Chroma, sentence-transformers, RAGAS, local NLI, and OpenAI can improve realism, but they are not required for public CI. The lightweight path uses deterministic lexical retrieval and fallback similarity logic so the tests remain accessible.

The non-goals are equally important. The project is not a full RAG platform. It is not tuned for maximum answer quality. It does not claim the NIST study corpus is a comprehensive benchmark. It also does not claim the fallback lexical oracle is equivalent to a trained NLI model. The project is a harness for testing and documenting RAG behavior.

Goal	Implementation choice
Public reproducibility	CI-safe dependencies and public corpus
Groundedness testing	NLI oracle with lexical fallback
Metamorphic behavior	Three paraphrases per seed question
Non-determinism tracking	Five repeated answer checks
Negative validation	Deliberately corrupted contexts

4. Corpus and Seed Question Design

The corpus is a compact study corpus derived from NIST AI Risk Management Framework themes. It includes sections such as Govern, Map, Measure, Manage, valid and reliable, safe, secure and resilient, accountable and transparent, privacy enhanced, fairness, documentation, monitoring, human oversight, and third-party risk. These sections create enough topical overlap to exercise retrieval while keeping the corpus small enough for manual review.

The seed bank contains 20 question-answer pairs. Each record has an id, question, gold answer, and supporting section. This design provides three pieces of evidence: the natural question, the intended answer meaning, and the section expected to support that answer. The supporting section is used to audit retrieval and to create corrupted-context cases.

The seed questions are short and practical. They ask what each function means, why context matters, what evidence measurement uses, how harmful bias is managed, why monitoring is needed, and what organizations should do about third-party components. This question style fits the corpus and avoids requiring external knowledge beyond the retrieved material.

The corpus is intentionally not scraped at runtime. It is committed as `data/nist_ai_rmf_corpus.md` so results do not drift when external web pages change. The project cites the official NIST AI RMF source as the conceptual basis [6], but the test corpus itself is a fixed local artifact.

Corpus section	Example test question theme
Govern	Accountability, policy, roles
Map	Context, stakeholders, harms
Measure	Evidence, validation, monitoring
Manage	Mitigation and risk response
Continuous monitoring	Drift and threshold review

5. Retrieval and Answer Pipeline

The retrieval layer uses a deterministic lexical store in the CI path. The store tokenizes the query and each chunk, scores overlap, and returns the top chunks. This is intentionally simple. It keeps the public test path fast and inspectable. When Chroma is installed, the project can persist a vector collection as an optional artifact, but the tests do not depend on it.

The answer pipeline matches each incoming question to the closest seed question. In the baseline path, it returns the seed gold answer. In an optional hosted-model path, it can ask a model to answer from supplied context. This design lets the harness test retrieval, metamorphic behavior, groundedness, and report generation without requiring a key.

The pipeline returns a structured `RAGAnswer` object containing the question, answer, contexts, matched seed id, and corruption flag. This shape makes downstream evaluation straightforward. The groundedness oracle receives the answer and contexts. The consistency scorer receives repeated answers. The metamorphic checker receives base and paraphrased answers.

This architecture separates test harness concerns from generation quality. A future implementation can replace the answer generator, retrieval store, or judge while keeping the same evaluation contract.

```
Corpus chunks -> lexical or vector store -> retrieved contexts
Seed question -> answer generator -> answer
Answer + contexts -> groundedness oracle
Question paraphrases -> repeated answers -> metamorphic relation check
```

The minimal pipeline is a strength for this case study. It makes every result explainable. If a case passes, the supporting context and matched seed are visible. If a case fails, the artifact shows which question, answer, and contexts were involved.

6. Groundedness Oracle

Groundedness means that the answer is supported by the retrieved context. In this project, retrieved context is treated as the evidence boundary. If the answer asserts facts that do not appear in or follow from the context, the answer should be flagged as ungrounded.

The local oracle supports two paths. The stronger path uses a sentence-transformers CrossEncoder for NLI when the dependency and model are available. The fallback path computes token overlap between the answer and context. The fallback is not a semantic proof, but it is deterministic and useful for CI. The report always records which provider was used.

The optional hosted judge path can ask a model to decide whether the answer is grounded in the context through the Responses API [7]. This path is useful for richer experiments, but it is not required for public tests. Keeping it optional avoids leaking secrets and keeps CI predictable.

The NLI framing follows the premise-hypothesis structure used in natural language inference. The retrieved context acts as the premise. The answer acts as the hypothesis. SNLI helped popularize large-scale entailment modeling for this type of relationship [5]. The project uses that framing in a practical test harness rather than claiming formal logical entailment.

The oracle returns a structured result:

Field	Meaning
grounded	Boolean decision
score	Provider-specific support score
reasoning	Short explanation or fallback note
provider	NLI, lexical fallback, or hosted judge

This structure keeps the report transparent. A pass/fail label alone is not enough; the provider and score explain how the decision was made.

7. Metamorphic Test Design

Metamorphic testing is useful when exact expected outputs are unavailable or too narrow. The idea is to transform the input while preserving a property that should remain stable. In this project, the transformation is paraphrasing. If the user asks the same question in three semantically equivalent ways, the answer should remain semantically equivalent.

The harness generates three paraphrases per seed question. If an OpenAI key is available, a hosted model can generate paraphrases. Otherwise, deterministic templates generate them. The template path is less linguistically rich but fully reproducible.

For each seed question, the pipeline records the base answer and each paraphrased answer. It computes semantic similarity between the base and paraphrased answers. When sentence-transformers is installed, the project uses a local embedding model. Otherwise, it uses a deterministic lexical fallback. Sentence-BERT provides the relevant sentence-embedding background [4].

The current violation threshold is intentionally visible in code. A paraphrase is marked violated when similarity drops below the chosen threshold. This threshold is not universal. It is a harness parameter that should be calibrated for a real domain.

Relation	Expected property
Seed question -> paraphrase 1	Answer meaning preserved
Seed question -> paraphrase 2	Answer meaning preserved
Seed question -> paraphrase 3	Answer meaning preserved

Metamorphic tests are not a replacement for human evaluation. They are a scalable way to expose instability. If a RAG pipeline answers paraphrases inconsistently, that is a useful regression signal even when no single exact answer string is required.

8. Consistency Scoring

RAG systems can vary from run to run. Variation is not automatically bad. A system may produce two different phrasings that mean the same thing. The problem is uncontrolled semantic drift. MetaRAG-Test measures that drift with repeated-answer consistency.

For each seed question, the harness runs the query five times and compares the repeated answers with the first answer. It records mean similarity and variance. The variance is the flakiness signal: low variance means repeated outputs are stable under the chosen comparison function; high variance means the system is changing meaning or wording enough to require review.

In the deterministic baseline, variance is 0.00 because the seed answer fallback returns the same answer. That is expected. The value of the mechanism appears when a hosted generator is enabled. Then the same metric can measure whether prompt or retrieval changes introduce more answer instability.

The consistency metric should be interpreted with the answer-generation mode. A deterministic fallback establishes the harness baseline. A hosted model run measures provider and prompt behavior. A local model run measures local runtime behavior. The report records enough metadata to make those distinctions clear.

Repeated answers	Mean similarity	Variance	Interpretation
Same answer each run	1.00	0.00	Stable deterministic path
Different wording, same meaning	High	Low	Acceptable variation
Different meaning	Low	High	Review required

This design is intentionally simple and explainable. More advanced scoring could include entailment pairs, rubric judging, or human labels, but the current metric is enough to expose whether repeated runs are stable.

9. Corrupted-context Validation

Groundedness oracles can look strong if they only see clean cases. A stronger validation includes negative controls. MetaRAG-Test deliberately corrupts retrieved context for each seed question by replacing the expected supporting section with unrelated sections. The answer remains the seed gold answer. A groundedness oracle should now flag the answer as unsupported because the retrieved evidence no longer contains the needed support.

This test is important because hallucination detection is not proven by a clean pass. A clean pass only shows that a supported answer was accepted. The corrupted-context case checks whether unsupported answers are rejected.

The current evaluation reports a 100.00 percent hallucination catch rate under the deterministic fallback. That result should be read in context. The corpus and answers are controlled, and the lexical fallback sees clear support differences. In a larger corpus with overlapping terminology, this metric would likely be lower and more informative.

The corrupted-context design is still valuable because it creates a reusable test pattern. Teams can build similar negative controls for their own RAG systems:

1. Keep the user question fixed.
2. Replace supporting context with plausible but wrong context.
3. Generate or reuse the answer.
4. Require the groundedness oracle to reject the answer.
5. Record false accepts as high-priority failures.

This pattern turns hallucination risk into a regression test rather than a vague quality concern.

10. Results and Interpretation

The current baseline evaluation produced the following summary:

Metric	Value
Seed questions	20
Groundedness accuracy	100.00%
Hallucination catch rate	100.00%
Metamorphic violation rate	0.00%
Mean flakiness variance	0.00
RAGAS proxy agreement	100.00%

These results show that the harness is internally consistent over the controlled corpus. Clean answers pass. Corrupted-context answers fail. Paraphrase checks preserve meaning. Repeated deterministic answers are stable. The RAGAS-style proxy agrees with the local groundedness decision.

The results should not be overread. The corpus is compact. The baseline answer generator returns seed answers. The lightweight fallback metrics are intentionally deterministic. A larger evaluation with hosted generation, noisy retrieval, and human labels would produce a more demanding result. The current value is that the harness structure is present and measurable.

The dashboard in `docs/index.html` summarizes the same metrics visually. The JSON artifact in `artifacts/evaluation.json` contains the full case-level evidence, including clean cases, corrupted cases, metamorphic cases, and consistency rows. Keeping both human-readable and machine-readable outputs makes the project suitable for review and extension.

The strongest engineering conclusion is that RAG quality gates should include negative controls and metamorphic checks. Exact gold-answer matching alone would miss the core groundedness problem.

11. Reproducibility and Operations

The public path requires only the CI dependency set:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-ci.txt
pytest
python scripts/run_evaluation.py
python scripts/build_case_study_pdf.py
```

The full local path installs optional RAG and model dependencies:

```
pip install -r requirements.txt
$env:OPENAI_API_KEY="<your-openai-api-key>"
$env:OPENAI_MODEL="gpt-5.5"
python scripts/run_evaluation.py --use-openai-answers --include-openai-judge
```

Operationally, the harness supports three modes. The first mode is deterministic CI: fast, cheap, and stable. The second mode is local extended evaluation with Chroma, sentence-transformers, RAGAS, and NLI models. The third mode is hosted-model evaluation with external generation and judging. Each mode should record which providers were active.

For a team setting, the recommended pattern is to run deterministic tests on every commit, run extended local or hosted evaluation on a schedule, and require review when groundedness, hallucination catch rate, or metamorphic stability drop below threshold. The thresholds should be domain-specific. A safety-critical system would require stricter review than an internal knowledge assistant.

The important operational principle is artifact retention. Keep the question, answer, contexts, scores, and provider metadata. Without those artifacts, a RAG score is hard to debug.

12. Limitations, Risks, and Future Work

The main limitation is scale. The corpus has enough structure for a case study, but it is not a broad benchmark. A larger corpus would introduce chunking tradeoffs, ambiguous retrieval, overlapping policy language, and multi-hop evidence. Those issues are useful future work.

The second limitation is the deterministic answer fallback. It makes the harness reproducible but underrepresents generation errors. Hosted or local generation should be used when the goal is to evaluate a complete RAG stack. The current baseline is best read as a test-harness validation.

The third limitation is oracle strength. The lexical fallback is transparent but shallow. The NLI path is stronger when the model is available, but NLI models can still fail on domain-specific wording, long contexts, and compositional claims. Hosted judges can provide richer reasoning, but they add cost and non-determinism.

Future work should add larger corpora, more relation types, multi-turn consistency, irrelevant-context invariance, answer abstention tests, retrieval recall checks, and human-labeled evaluation sets. It should also compare the local NLI oracle, hosted judge, and RAGAS metrics on the same cases.

Despite those limitations, MetaRAG-Test demonstrates a practical engineering pattern. Treat RAG testing as a set of relations and evidence checks: clean context should ground the answer, corrupted context should be rejected, paraphrases should preserve meaning, and repeated runs should remain stable enough for the system's risk level.

13. References

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [2] S. Es et al., "RAGAS: Automated Evaluation of Retrieval Augmented Generation," EACL Demo, 2024. <https://aclanthology.org/2024.eacl-demo.16/>
- [3] S. Hyun, M. Guo, and M. A. Babar, "METAL: Metamorphic Testing Framework for Analyzing Large-Language Model Qualities," 2023. <https://arxiv.org/abs/2312.06056>
- [4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP-IJCNLP, 2019. <https://arxiv.org/abs/1908.10084>
- [5] S. R. Bowman et al., "A Large Annotated Corpus for Learning Natural Language Inference," EMNLP, 2015. <https://aclanthology.org/D15-1075/>
- [6] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
- [7] OpenAI, "Responses Overview - OpenAI API Reference." <https://developers.openai.com/api/reference/responses/overview/>
- [8] RAGAS Documentation, "Faithfulness metric." https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/faithfulness/